Software Engineering — Assignment #04

Smart Campus Room Availability Checker

Student: Serik Dias

Student ID: 24B032021

Date: 6 November 2026

1. Scope and implementation approach

The application checks whether a fictional room is suitable for a requested group and time. It does not create or confirm bookings. The demo uses a fixed current time of 09:00 (540 minutes), and all room and booking data are local fictional fixtures.

The supplied Assignment #03 ZIP contained `booking.py`, `test_booking.py`, documentation and evidence, but no original web source export. The assignment allows an equivalent minimal reconstruction. The original Python function and tests were preserved under `baseline/` and at the project root. A modular responsive frontend was reconstructed for the new multi-room scope.

Five rooms are included. Each record has `id`, `name`, `capacity`, `equipment`, `blocked` and an independent `bookings` list. The UI filters by capacity and all selected equipment, shows an empty state when no room matches, recalculates availability when the selected room changes, and accepts 24:00 as the end-of-day endpoint.

2. Task 01 — Codebase analysis

Maintainability risks found in the supplied baseline

#	Risk	Evidence (file / function)	Why it matters
1	One function owns all availability decisions	`booking.py`, `can_book()`	Input validation, day bounds, blocked status and overlap rules are all in one function. Small today, but room selection and filtering would make it grow.
2	Positional arguments and loosely shaped booking intervals	`booking.py`, signature `can_book(start, end, now, blocked, existing)` and loop `for busy_start, busy_end in existing`	Callers must remember argument order and interval shape. A room object makes the source of bookings explicit.
3	Only a boolean is returned, so no reason can be shown	`booking.py`, every `return False` / `return True` in `can_book()`	AC6 requires clear feedback. A UI cannot tell the user *why* a request failed; it needs a result object with a reason.
4	Magic numbers inside the rule	`booking.py`, `can_book()`: literal `1440` (day bound) and `120` (duration limit)	Limits are not named, so they are easy to change in one place and miss in another once messages or a UI repeat them.

The original Assignment #03 web source was not in the supplied archive, so the baseline analysis covers the Python source only; the frontend was reconstructed (see Section 1).

Prioritized improvement plan (recorded before implementation)

1. Preserve and rerun the original Python tests before changing behavior.
2. Define a room data model with a separate booking list for each room.
3. Separate pure time/overlap rules from capacity/equipment filtering.
4. Add an application service that always resolves the selected room by ID before checking availability.

5 Keep DOM rendering and event listeners in a UI module; use accessible status feedback and responsive CSS.

6 Add regression, filter, boundary and integration tests; record actual commands and results.

AI suggestion rejected or modified

A suggestion to derive the current time from the user's computer was rejected. The assignment requires a fixed current time, so the app uses 09:00. A suggestion to remove blocked rooms from the visible results was modified: blocked rooms remain visible and are labelled, while the availability rule always returns unavailable. This makes F3 observable and keeps filtering focused on the two specified requirements: capacity and equipment.

3. Task 02 — Feature extension

The room catalogue contains A101, B202, C303, D404 and E505. Capacity filtering uses `room.capacity >= attendees`. Equipment filtering uses `every`, so a room must contain all selected items. Blocked rooms may appear in the matching catalogue but cannot return an available result.

The availability check uses the room found by the currently selected ID. It does not reuse another room's booking data. When the room selector changes, the result is recalculated immediately. When no room matches, the UI displays a clear empty-state message and disables the selector.

The preserved rules are:

- Start/end are integer minute values within the same day, with `0 <= start < end <= 1440`.
- Start must be strictly later than 09:00.
- Maximum duration is 120 minutes inclusive.
- A blocked room is unavailable.
- Intervals are half-open `[start, end)`, so touching endpoints are allowed.
- Existing booking data is read-only; no booking is created.

4. Task 03 — Refactoring

Before: supplied baseline

```
test_booking.py
     |
     v
booking.py
  can_book()
  - input validation
  - time boundaries
  - blocked check
  - overlap check
  - boolean result
```

After: modular application

```
index.html + styles.css
         |
         v
    src/ui/app.js  ------------------- UI rendering/events
         |
         v
src/application/roomChecker.js ------- selected-room orchestration
      /             \
     v               v
roomFilters.js   availability.js ------- pure domain rules
      ^               ^
       \             /
        src/data/rooms.js -------------- fictional room records
```

Three key design decisions:

1. **Separate filtering from availability.** Capacity/equipment requirements decide which rooms are candidates. The availability module separately checks time, blocked status and overlap. This prevents the filter from accidentally replacing the booking rules.
2. **Resolve the selected room by ID in one application service.** `checkSelectedRoom()` finds the room first and passes that exact room to the pure availability function. The integration test demonstrates that A101 and B202 produce different outcomes for the same time because their booking lists differ.
3. **Named constants for rule limits.** `MAX_DURATION_MINUTES` and `END_OF_DAY_MINUTES` are defined once in `availability.js`. Validation, time formatting, time parsing and the user-facing messages all use them; the start-time message is built from the `now` argument instead of repeating "09:00".

The original Python implementation remains available as a regression baseline. The new frontend logic is organized into data, domain, application and UI responsibilities.

5. Task 04 — Automated testing

Commands executed

```
npm install                          # installs jsdom, used only by tests/ui.test.js
npm test                             # runs node --test
python -m unittest -v test_booking.py
```

Actual results

Suite	Result	Notes
JavaScript Node test runner	34/34 passed	17 availability/time tests, 6 filtering, 5 integration/catalogue/asset checks, 6 jsdom UI-behavior tests
Python legacy regression suite	4/4 unittest methods passed	The main acceptance method runs 36 subcases; other methods cover floats, nested lists and tuples

Full console output is saved in `evidence/node_tests.txt` and `evidence/python_regression_tests.txt`, each starting with the command that produced it.

The JavaScript tests explicitly cover exactly 120 vs 121 minutes, touching vs overlapping intervals, exact vs insufficient capacity, all-selected-equipment behavior, blocked rooms, empty results, selected-room switching and unchanged booking data. `tests/ui.test.js` loads `index.html` in jsdom, runs `src/ui/app.js` and drives the form with events: initial render, room switching (F5), empty state with disabled selector (F4), equipment filtering (F2), 120/121 minutes in the UI, and the HH:MM hint for malformed time input.

jsdom simulates the DOM, not a real browser: CSS layout, the responsive breakpoint on real screens, keyboard-only use and screen readers are not tested automatically. One additional test checks that the HTML/CSS contain the module entry point, live region and a `max-width: 640px` breakpoint.

6. Task 05 — Quality evaluation

Measure	Baseline	Final version	How obtained
Automated tests passed / total	Python: 4/4 methods (36 subcases); no JavaScript tests	Python: 4/4 methods; JavaScript: 34/34	Ran the commands in Section 5; output saved in `evidence/`
Hard-coded rule-limit literals in JS rule code (`1440`, `120`, `"09:00"` in `src/domain/` and `src/ui/`)	Python `booking.py`: 2 (`1440`, `120`). First reconstructed JS draft of `availability.js`: 7	2 (the two named constant definitions)	`grep -nE "1440|120|09:00" src/domain/*.js src/ui/app.js src/application/*.js`. The JS draft count was taken on that file before this change and is not included in the archive.
Duplicated logic locations removed	n/a	5 repeated literal uses removed (7 → 2)	Difference of the counts above. This counts repeated constants, not repeated rule algorithms; no duplicated rule block existed in `can_book()`.

Measure	Baseline	Final version	How obtained
Code structure	1 Python module, 37 lines, 1 function	5 JS modules, 183 lines: `data/rooms.js` (data), `domain/availability.js` (time/booking rules), `domain/roomFilters.js` (filtering), `application/roomChecker.js` (selected-room orchestration), `ui/app.js` (DOM)	`wc -l`; responsibilities from code review
Remaining limitations	No reasons returned; no room model	See below	Scope and verification record

Remaining limitations:

- The availability rules now exist twice: `booking.py` (kept only as the regression baseline) and `availability.js` (used by the app). They are verified separately and are not generated from one source.
- `index.html` still contains "09:00" twice as initial placeholder text; `app.js` overwrites both from `DEMO_NOW_MINUTES` on load.
- UI tests run in jsdom. No real-browser, real-device, keyboard-only or screen-reader testing was done.
- All data are fictional and in memory; the current time is fixed at 09:00.

7. Task 06 — AI reflection (200–300 words)

I used ChatGPT (GPT-6) on 6 November 2026 to inspect the supplied Assignment #03 archive, plan the changes and review the implementation. I provided the Assignment #04 requirements, the repository file list, the original Python function and its tests, and the fact that the original frontend source was missing. I asked for a small, modular implementation rather than a complete rewrite of the preserved baseline.

The most useful suggestions were to keep room data separate from booking rules, filter rooms with an all-selected-equipment check, and resolve the selected room by its ID before checking availability. I did not follow the suggestion to use the computer's live clock because the assignment specifies a fixed demo time. I kept blocked rooms visible so the interface can show why a room is unavailable.

I personally checked the room records, the overlap expression, the capacity and equipment predicates, the selected-room lookup and the test fixtures. A final review found real problems: malformed time input showed a "whole-number minutes" message, the result icon was a check mark even when unavailable, and rule limits were repeated as literals. I fixed these with named constants, an HH:MM hint and a state-dependent icon, and added jsdom tests that drive the real UI code. One of my new UI tests failed at first; the cause was my test, not the app, because the app correctly kept the previously selected room. Final results: 34/34 JavaScript tests and 4/4 Python test methods. Browser, device and assistive-technology testing remain undone, and the rules still exist in both Python and JavaScript. Next, I would add real-browser tests and generate both implementations from one specification.

Submission note: The report and evidence describe only checks actually run for this final reconstructed project. No deployment, live API, database or email submission is claimed.